Units 11–12: Balanced Trees & B-Trees (Week 9)

CPTR 242 — Sequential and Parallel Data Structures and Algorithms

Walla Walla University

Part 1: AVL Trees

Sections 11.1 – 11.4

🤔 A plain BST can degenerate into a linked list.

Insert 1, 2, 3, 4, 5 in order and every node goes right.

Height = n, all operations = O(n).

What if the tree automatically fixed itself after every insert and delete to stay short?

11.1 The AVL Tree

An **AVL tree** is a BST that keeps itself balanced automatically. It was the first self-balancing BST, invented in 1962.

The AVL rule: at every node, the heights of the left and right subtrees differ by **at most 1**.

```
    30          ← balanced: left height 1, right height 1
   /  \
  20   40
```

This one rule guarantees the tree height stays at **O(log n)** — so search, insert, and delete are always **O(log n)**, no matter what order values are inserted.

11.1 Balance Factor

Each node tracks a **balance factor** = height(right subtree) – height(left subtree).

Balance factor	Meaning
–1, 0, +1	✅ Balanced — no action needed
–2	❌ Left-heavy — needs fixing
+2	❌ Right-heavy — needs fixing

```
      30  (bf = 0)
     /  \
   20    40        20 has bf = 0, 40 has bf = 0
  /
10  (makes 20's bf = -1, 30's bf = -1 — still OK)
```

A balance factor of ±2 means the AVL rule is violated and the tree needs a **rotation**.

🤔 The tree is lopsided. What's the least disruptive way to fix it?

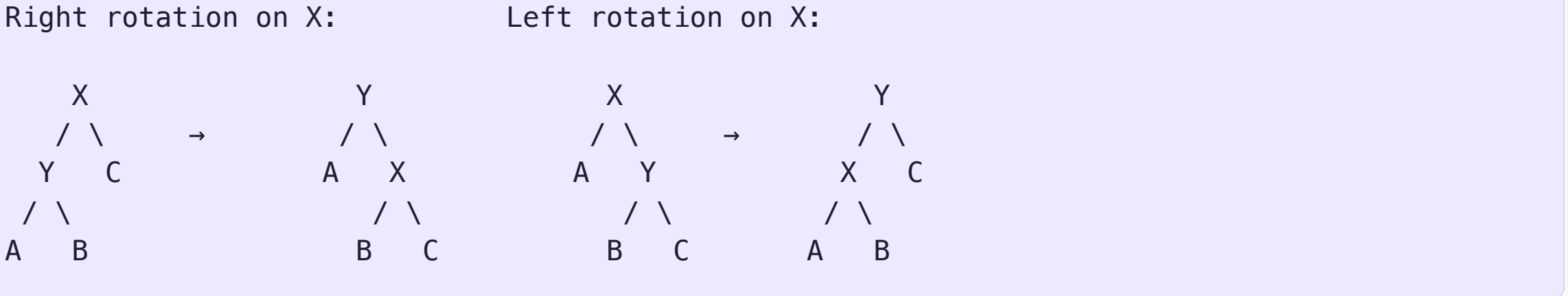
You can't just rearrange nodes arbitrarily — the BST ordering must be preserved.

What operation reshuffles the shape while keeping all values in the correct order?

11.2 Rotations: The Basic Idea

A rotation is a local restructuring operation on a BST. A rotation may change subtree heights and balance, while preserving BST order.

There are only **two directions**: rotate left and rotate right.



Think of it as: **the child rises, the parent falls**. The "inner" subtree (B) moves across.

> BST order is preserved: everything left of X stays left, everything right stays right.

11.2 Four Rotation Cases

When balance factor reaches ±2, exactly one of four cases applies:

Case	Shape	Fix
Left-Left (LL)	Left child, left grandchild	Rotate right at node
Right-Right (RR)	Right child, right grandchild	Rotate left at node
Left-Right (LR)	Left child, right grandchild	Rotate left at child, then right at node
Right-Left (RL)	Right child, left grandchild	Rotate right at child, then left at node

LL and RR need **one** rotation. LR and RL need **two**.

11.2 LL Case: One Right Rotation

```
Insert 30, 20, 10:

Step 1: insert 30          Step 2: insert 20          Step 3: insert 10
     30                         30                         30  ← bf = −2 ❌
                                /                          /
                              20                         20
                                                         /
                                                       10
```

Fix: **rotate right** at 30. Tree is balanced again.

```
Before:    30          After right rotation:    20
           /                                   /  \
         20                                  10    30
         /
       10
```

11.2 LR Case: Two Rotations

```
Insert 30, 10, 20:

     30   ← bf = −2 ❌
     /
   10
     \
     20   ← left child has right grandchild = LR case
```

Fix: **rotate left at 10**, then **rotate right at 30**.

```
Step 1 — left rotate at 10:    Step 2 — right rotate at 30:
     30                              20
     /                              /  \
   20              →              10    30
   /
 10
```

The two-rotation fix always "straightens" the zigzag before rotating the parent.

11.3 AVL Insertion

Insert exactly like a plain BST, then fix on the way back up.

```
1. Insert the new node as a BST leaf
2. Walk back up to the root, updating balance factors
3. At the first node with balance factor ±2:
     apply the correct rotation (LL, RR, LR, or RL)
4. Done — one rotation (or double rotation) is always enough
```

Only the **first** unbalanced ancestor needs fixing. After the rotation, heights above it are restored automatically.

Cost: O(log n) for the insert + O(log n) to walk back up = **O(log n) total**.

11.4 AVL Removal

Remove exactly like a plain BST (leaf / one child / two children), then fix on the way back up.

```
1. Remove the node using standard BST removal
2. Walk back up to the root, updating balance factors
3. At every node with balance factor ±2: apply a rotation
4. Unlike insertion, may need rotations all the way to the root
```

The key difference from insertion: removal can require **multiple rotations** up the tree, not just one.

Cost: still **O(log n)**, at most O(log n) rotations, each O(1).

Part 2: Red-Black Trees

Sections 11.6 – 11.9

🤔 AVL trees are perfectly balanced but require careful bookkeeping.

Every insert and delete may trigger rotations and balance-factor updates all the way to the root.

Is there a balanced BST that is slightly less strict — but simpler to implement and faster in practice?

11.6 The Red-Black Tree

A **red-black tree** is a BST where every node is colored **red** or **black**, and four rules keep the tree balanced.

The four rules:

1. Every node is red or black
2. The root is black
3. No red node has a red child *(no two reds in a row)*
4. Every path from root to a null pointer passes through the **same number of black nodes**

```
        10 (B)
       /      \
    5 (R)    15 (R)
   / \        / \
3(B) 7(B) 12(B) 20(B)
```

11.6 Why the Rules Work

Rule 4 says every root-to-null path has the same **black height**. This means:

- The shortest possible path is all black nodes
- The longest possible path alternates red-black-red-black

So the longest path is **at most twice** the shortest path, giving height $\leq$ **2 $\log_2$(n+1)**.

This is slightly looser than AVL's strict ±1 rule — but it means fewer rotations are needed on insert and delete, which is why `std::map` and `std::set` use red-black trees.

> AVL trees are better for **read-heavy** workloads (more balanced = faster search). Red-black trees are better for **write-heavy** workloads (fewer rotations on insert/delete).

11.7 Red-Black Rotations

Red-black trees use the same left and right rotations as AVL trees. The mechanics are identical, the difference is **when** they trigger.

After a rotation, node colors may also need to be swapped to maintain the four rules.

The rotation fixes shape; the recolor fixes color.

11.8 Red-Black Insertion

Insert as a BST leaf, color the new node **red**, then fix violations.

Why red? Adding a red node cannot violate Rule 4 (black height unchanged). It may violate Rule 3 (red child of red parent). That violation is easier to fix.

Three cases to fix a red-red violation, depending on the **uncle** node's color:

Uncle color	Fix
Red	Recolor parent + uncle black, grandparent red; move up
Black (outer)	One rotation + recolor
Black (inner)	Two rotations + recolor

Red-black insertion requires at most 2 rotations, similar to AVL insertion, but often performs fewer structural changes because many violations are resolved with recoloring alone.

11.8 Insertion Example

Insert 10, 20, 30 into an empty red-black tree:

```
Insert 10:    Insert 20:       Insert 30 — violation!
  10(B)         10(B)              10(B)
                  \                  \
                  20(R)              20(R)  ← red parent
                                       \
                                       30(R) ← red child ❌
```

Uncle of 30 is null (counts as black). This is the "outer" case → rotate left at 10, recolor:

```
      20(B)
     /    \
  10(R)   30(R)
```

11.9 Red-Black Removal

Removal is the most complex operation, more cases than insertion.

High-level idea:

1. Remove using standard BST removal
2. If the removed node was **black**, the black-height rule is violated, one path has one fewer black node
3. Fix by: recoloring or rotating

The fix propagates up the tree in the worst case, but always terminates in **O(log n)** steps with at most **3 rotations** total.

> Red-black deletion is rarely implemented from scratch. In practice you use `std::map` / `std::set`, which handle it internally. Understanding the concept matters more than memorizing every case.

AVL vs. Red-Black: Summary

Property	AVL Tree	Red-Black Tree
Balance guarantee	Height ≤ 1.44 $\log_2 n$	Height ≤ 2 $\log_2 n$
Search	Faster (more balanced)	Slightly slower
Insert rotations	Up to O(log n)	At most 2
Delete rotations	Up to O(log n)	At most 3
Used in	Read-heavy, databases	`std::map`, `std::set`, Linux kernel

Both guarantee **O(log n)** for all operations. The difference is constant factors and implementation complexity.

Part 3: B-Trees

Sections 12.1 – 12.5

🤔 AVL and red-black trees work great in RAM. But what about on disk?

Reading one node from disk takes ~10ms. Reading from RAM takes ~100ns: 100,000× faster.

If each disk read fetches one tree node, how would you redesign the tree to minimize the number of disk reads?

12.1 The B-Tree

A **B-tree** solves the disk-access problem: storing **many keys per node** rather than one.

- Each node holds up to **t** keys and up to **t+1** children
- All leaf nodes are at the **same depth** (perfectly balanced by height)
- Every non-root node is **at least half full**

```
        [ 20 | 40 ]
       /     |     \
[10|15]  [25|30]  [45|50]
```

One disk read fetches an entire node with many keys. A tree with millions of keys may need only **3–4 levels**, meaning 3–4 disk reads to find anything.

> B-trees are the data structure inside every major database (PostgreSQL, MySQL, SQLite) and every filesystem (NTFS, ext4, APFS).

12.1 B-Tree vs. Binary Tree Height

With n keys and each node holding up to t keys:

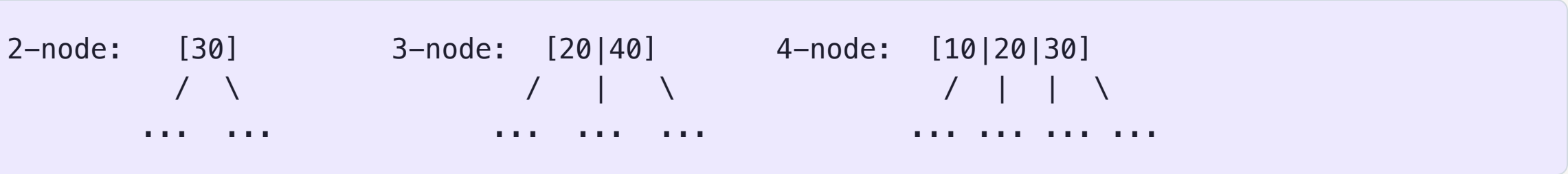
n	Binary BST height	B-tree height (t=500)
1,000	~10	2
1,000,000	~20	4
1,000,000,000	~30	6

Each B-tree level corresponds to one disk read. The difference between 30 disk reads and 6 disk reads is dramatic when each read costs milliseconds.

12.2 2-3-4 Tree

A **2-3-4 tree** is a B-tree where every node holds **1, 2, or 3 keys** (and 2, 3, or 4 children). It is the simplest B-tree to study.

Node type	Keys	Children
2-node	1 key	2 children
3-node	2 keys	3 children
4-node	3 keys	4 children



All leaves are at the same level. The tree is always **perfectly balanced by height**.

12.2 2-3-4 Tree: Search

Search works exactly like BST search, but at each node you compare against **multiple keys**.

```
Search for 35 in:
          [ 20 | 40 ]
         /     |     \
  [10|15]  [25|30]  [45|50]

At root: 35 > 20, 35 < 40 → go to middle child
At [25|30]: 35 > 25, 35 > 30 → go to right child
At [45|50]: 35 < 45 → go to left child → null → not found
```

At each node, scan the keys left to right to decide which child pointer to follow. With t keys per node, each node takes O(t) to scan — but t is a constant, so overall cost is **O(log n)**.

12.3 2-3-4 Tree: Insertion

Always insert at a **leaf**. The challenge: what if the leaf is a 4-node (already has 3 keys)?

Solution: split 4-nodes on the way down before reaching the leaf.

When you encounter a 4-node during the search:

1. **Split** it: push the middle key up to the parent
2. The 4-node becomes two 2-nodes

```
Before split:          After split:
  [ 10 | 20 | 30 ]       parent gets 20
                           /          \
                         [10]         [30]
```

By the time you reach the leaf, it is guaranteed to have room. Insert directly.

12.3 Insertion Trace

Insert 25 into this tree:

```
        [ 20 | 40 | 60 ]  ← 4-node: split first!
```

Split: push 40 up, but this is the root — create a new root:

```
          [40]
          /  \
       [20]  [60]
```

Now insert 25: 25 < 40 → go left to [20]. 25 > 20 → insert in right position:

```
            [40]
           /    \
       [20|25]  [60]
```

12.4 Deletion: Rotation and Fusion

When **removing** from a 2-3-4 tree, there are 2 approaches to rebalance:

Rotation: borrow a key from an adjacent sibling through the parent:

Fusion: merge the current node with a sibling and the separator key from the parent:

Rotation is preferred (parent stays the same size). Fusion shrinks the parent and may propagate upward.

B-Tree Family: The Big Picture

Structure	Node size	Use case
BST	1 key	In-memory, simple
AVL tree	1 key	Read-heavy, in-memory
Red-black tree	1 key	`std::map`, write-friendly
2-3-4 tree	1–3 keys	Teaching B-trees
B-tree (t=100–500)	Up to t keys	Database indexes
B+ tree	Up to t keys (data only at leaves)	Most real databases

The **B+ tree** variant stores all data in leaves and uses internal nodes only for routing. This makes range queries (find all keys between 10 and 50) very fast, just scan the leaf level.

Unit 11–12 Summary

Structure	Height	Insert rotations	Delete rotations	Best for
Plain BST	O(n) worst	0	0	Simple cases
AVL tree	≤ 1.44 log n	O(log n)	O(log n)	Read-heavy
Red-black tree	≤ 2 log n	≤ 2	≤ 3	`std::map` / `std::set`
2-3-4 / B-tree	O(log_t n)	O(log n) splits	O(log n) fusions	Disk / databases

The theme of this unit: the plain BST's O(n) worst case is unacceptable in production. Every structure here pays a small constant overhead on insert and delete to guarantee O(log n) always.

Lab 9 ! 😄